



Correction de TP1 – GESTIONNAIRE DE TACHES EN CONSOLE

Duree : 45 minutes

Objectif : Maitriser les classes, le null safety, les collections et les methodes fonctionnelles en Dart.

SOMMAIRE DES ETAPES

Etape	Contenu	Duree
Etape 1	Modeliser la classe Tache	15 min
Etape 2	Creer le GestionnaireTaches	15 min
Etape 3	Creer le menu interactif	15 min

ETAPE 1 : MODELISER LA CLASSE TACHE (15 min)

1.1 Ce que vous allez creer

Une classe `Tache` avec :

- `id` (String, final, requis)
- `titre` (String, requis)
- `description` (String?, nullable)
- `estTerminee` (bool, default false)
- `dateCreation` (DateTime, final, requis)
- `priorite` (enum Priorite : basse, moyenne, haute)



1.2 Le code de l'Etape 1

```
dart

// =====
// IMPORT : dart:io permet de lire les entrees clavier (stdin)
// =====
import 'dart:io';

// =====
// ENUM PRIORITE
// =====
// Un enum (enumeration) est un type qui ne peut prendre
// que des valeurs predefinies.
// Ici : Priorite.basse, Priorite.moyenne, Priorite.haute
enum Priorite {
    basse,    // Valeur 1
    moyenne,  // Valeur 2
    haute,    // Valeur 3
}

// =====
// CLASSE TACHE
// =====
class Tache {
    // -----
    // LES CHAMPS (proprietes de la classe)
    // -----
    final String id;           // final = ne peut plus etre modifie apres creation
    final String titre;        // String non nullable = obligatoire
    final String? description; // String? nullable = peut etre null
    final bool estTerminee;    // bool = true (terminee) ou false (en cours)
    final DateTime dateCreation; // DateTime = date et heure de creation
    final Priorite priorite;    // enum = basse, moyenne ou haute

    // -----
    // CONSTRUCTEUR PRINCIPAL
    // -----
    // Les parametres sont NOMMES (avec des accolades {})
    // required = Le parametre est obligatoire
    // this.description = optionnel (peut etre omis, sera null)
    // this.estTerminee = false = valeur par default si non fourni
```



```
Tache({
    required this.id,
    required this.titre,
    this.description,
    this.estTerminee = false,
    required this.dateCreation,
    required this.priorite,
});

// -----
// FACTORY CONSTRUCTOR : creer une Tache depuis un Map JSON
// -----
// factory = constructeur special qui peut retourner une instance
// Prend un Map<String, dynamic> (cle = String, valeur = n'importe quel type)
factory Tache.fromJson(Map<String, dynamic> json) {
    return Tache(
        id: json['id'] as String,           // as String = conversion de type
        titre: json['titre'] as String,
        description: json['description'] as String?, // as String? = peut etre null
        estTerminee: json['estTerminee'] as bool? ?? false, // ?? = valeur par defau
t
        dateCreation: DateTime.parse(json['dateCreation'] as String), // parse = con
vertir String en DateTime
        priorite: Priorite.values.firstWhere((p) => p.name == json['priorite']),
        // Priorite.values = liste [basse, moyenne, haute]
        // firstWhere = trouve le premier element qui correspond
        // p.name = Le nom de l'enum ("basse", "moyenne", "haute")
    );
}

// -----
// METHODE toJson() : convertir la Tache en Map pour le JSON
// -----
Map<String, dynamic> toJson() {
    return {
        'id': id,
        'titre': titre,
        'description': description,
        'estTerminee': estTerminee,
        'dateCreation': dateCreation.toIso8601String(), // convertir DateTime en Str
ing ISO
        'priorite': priorite.name, // .name = Le nom de l'enum en String
    };
}
```



```
};  
}  
  
// -----  
// METHODE toString() : affichage formate  
// -----  
// @override = on redefinit la methode toString() de Object  
// Affichage : [HAUTE] Acheter du Lait - En cours  
@override  
String toString() {  
    // Operateur ternaire : condition ? si_vrai : si_faux  
    final statut = estTerminee ? 'Terminee' : 'En cours';  
    // priorite.name.toUpperCase() = "haute" devient "HAUTE"  
    return '[' + priorite.name.toUpperCase() + ']' + $titre + ' - ' + statut;  
}  
}
```

1.3 Questions de comprehension de l'Etape 1

Question	Reponse
Q1 : Pourquoi <code>id</code> est-il <code>final</code> ?	Parce qu'un identifiant ne doit jamais changer apres creation
Q2 : Quelle est la difference entre <code>String</code> et <code>String?</code> ?	<code>String</code> ne peut jamais etre null, <code>String?</code> peut etre null
Q3 : A quoi sert <code>required</code> ?	A obliger le developpeur a fournir cette valeur
Q4 : Que fait le <code>factory</code> ?	Il cree une instance depuis des donnees JSON (Map)
Q5 : Que fait <code>?? false</code> ?	Si la valeur est null, on utilise <code>false</code> comme valeur par default



ETAPE 2 : CREER LE GESTIONNAIRE DE TACHES (15 min)

2.1 Ce que vous allez creer

Une classe `GestionnaireTaches` qui gere une liste de taches avec les methodes :

- `ajouter(Tache tache)`
- `supprimer(String id)`
- `marquerCommeTerminee(String id)`
- `listerToutes()` : trieés par date décroissante
- `listerParPriorite(Priorite p)`
- `listerTerminees()`
- `compterEnCours()`
- `rechercher(String terme)`

2.2 Le code de l'Etape 2

dart

```
// =====  
// CLASSE GESTIONNAIRE DE TACHES  
// =====  
class GestionnaireTaches {  
  // -----  
  // LISTE PRIVEE DES TACHES  
  // -----  
  // Le _ devant _taches rend la variable privée  
  // (accessible uniquement dans cette classe)  
  final List<Tache> _taches = [];  
  
  // -----  
  // METHODE : ajouter une tache  
  // -----  
  void ajouter(Tache tache) {  
    _taches.add(tache); // add() = ajoute a la fin de la liste  
    print('Tache ajoutée : $tache'); // $tache = interpolation (appelle toString())  
  }  
  
  // -----  
  // METHODE : supprimer une tache par son ID
```



```
// -----  
void supprimer(String id) {  
    // indexWhere = retourne l'index du premier element qui correspond  
    // Si aucun element ne correspond, retourne -1  
    final index = _taches.indexWhere((t) => t.id == id);  
  
    if (index != -1) {  
        // removeAt = supprime l'element a l'index donne  
        final tacheSupprimee = _taches.removeAt(index);  
        print('Tache supprimee : $tacheSupprimee');  
    } else {  
        print('Aucune tache trouvee avec l\'ID : $id');  
    }  
}  
  
// -----  
// METHODE : marquer une tache comme terminee (toggle)  
// -----  
void marquerCommeTerminee(String id) {  
    final index = _taches.indexWhere((t) => t.id == id);  
  
    if (index != -1) {  
        final ancienne = _taches[index];  
  
        // Les champs sont final → on cree une NOUVELLE tache  
        // avec le bool estTerminee INVERSE  
        _taches[index] = Tache(  
            id: ancienne.id,  
            titre: ancienne.titre,  
            description: ancienne.description,  
            estTerminee: !ancienne.estTerminee, // ! = negation (true devient false)  
            dateCreation: ancienne.dateCreation,  
            priorite: ancienne.priorite,  
        );  
        print('Statut mis a jour : ${_taches[index]}');  
    } else {  
        print('Aucune tache trouvee avec l\'ID : $id');  
    }  
}  
  
// -----  
// METHODE : lister toutes les taches (triees par date)
```



```
// -----  
List<Tache> listerToutes() {  
    // List.from = cree une COPIE de la liste (ne modifie pas l'originale)  
    final liste = List<Tache>.from(_taches);  
  
    // sort() = trie la liste en place  
    // b.dateCreation.compareTo(a.dateCreation) :  
    //   - retourne negatif si b < a  
    //   - retourne 0 si b == a  
    //   - retourne positif si b > a  
    // b avant a = tri DECROISSANT (les plus recentes d'abord)  
    liste.sort((a, b) => b.dateCreation.compareTo(a.dateCreation));  
  
    return liste;  
}  
  
// -----  
// METHODE : filtrer par priorite  
// -----  
List<Tache> listerParPriorite(Priorite p) {  
    // where() = filtre la liste  
    // Garde seulement les elements ou la condition est vraie  
    // toList() = convertit le resultat en List  
    return _taches.where((t) => t.priorite == p).toList();  
}  
  
// -----  
// METHODE : filtrer les taches terminees  
// -----  
List<Tache> listerTerminees() {  
    // where() garde seulement les taches ou estTerminee == true  
    return _taches.where((t) => t.estTerminee).toList();  
}  
  
// -----  
// METHODE : compter les taches en cours (non terminees)  
// -----  
int compterEnCours() {  
    // where() filtre les taches non terminees  
    // .length = Le nombre d'elements dans la liste resultante  
    return _taches.where((t) => !t.estTerminee).length;  
}
```



```
// -----  
// METHODE : rechercher dans le titre et la description  
// -----  
List<Tache> rechercher(String terme) {  
    // toLowerCase() = convertit en minuscules pour ignorer la casse  
    final termeLower = terme.toLowerCase();  
  
    return _taches.where((t) {  
        // contains() = verifie si la chaine contient le terme  
        final titreMatch = t.titre.toLowerCase().contains(termeLower);  
  
        // t.description?.toLowerCase() = appel securise (seulement si non null)  
        // ?? false = si description est null, on considere qu'il n'y a pas de match  
        final descMatch = t.description?.toLowerCase().contains(termeLower) ?? false  
    }  
    ;  
  
    // || = OU logique (titre match OU description match)  
    return titreMatch || descMatch;  
}).toList();  
}
```

2.3 Questions de comprehension de l'Etape 2

Question	Reponse
Q1 : Pourquoi la liste est-elle privée (<code>_taches</code>) ?	Pour empêcher la modification directe depuis l'extérieur. Seules les méthodes de la classe peuvent la modifier.
Q2 : Que fait <code>indexOfWhere()</code> ?	Elle retourne l'index du premier élément qui correspond à la condition, ou -1 si aucun ne correspond.



Question	Reponse
Q3 : Pourquoi creer une NOUVELLE tache pour <code>marquerCommeTerminee()</code> ?	Parce que les champs sont <code>final</code> (immuables). On ne peut pas modifier, donc on cree une copie avec la valeur inversee.
Q4 : Comment fonctionne <code>sort()</code> ?	Elle compare deux elements a la fois et les reordonne selon le resultat de la fonction de comparaison.
Q5 : Que fait <code>where()</code> ?	Elle filtre la liste et retourne seulement les elements qui satisfont la condition.
Q6 : Que fait <code>?.</code> (appel securise) ?	Si l'objet est null, l'expression retourne null sans planter. Sinon, elle appelle la methode normalement.

ETAPE 3 : CREER LE MENU INTERACTIF (15 min)

3.1 Ce que vous allez creer

Une fonction `main()` qui :

1. Instancie le gestionnaire
2. Pre-remplit 5 taches de demonstration
3. Affiche un menu avec 8 options
4. Boucle jusqu'a ce que l'utilisateur choisisse 8 (Quitter)

3.2 Le code de l'Etape 3

```
dart
```

```
// =====  
// POINT D'ENTREE DU PROGRAMME  
// =====  
void main() {
```



```
// Instancier Le gestionnaire
final gestionnaire = GestionnaireTaches();

// -----
// PRE-REPLISSAGE AVEC 5 TACHES DE DEMONSTRATION
// -----
final tachesDemo = [
  Tache(
    id: '101',
    titre: 'Apprendre Dart',
    description: 'Maitriser les bases du langage',
    dateCreation: DateTime.now().subtract(const Duration(days: 2)),
    // subtract = soustrait 2 jours a la date actuelle
    priorite: Priorite.haute,
  ),
  Tache(
    id: '102',
    titre: 'Faire les courses',
    description: 'Acheter du lait et du pain',
    dateCreation: DateTime.now().subtract(const Duration(days: 1)),
    priorite: Priorite.moyenne,
  ),
  Tache(
    id: '103',
    titre: 'Reviser Flutter',
    description: null, // Tache SANS description (test du nullable)
    dateCreation: DateTime.now(),
    priorite: Priorite.haute,
  ),
  Tache(
    id: '104',
    titre: 'Appeler le dentiste',
    description: 'Prendre rendez-vous',
    dateCreation: DateTime.now().subtract(const Duration(days: 3)),
    priorite: Priorite.basse,
    estTerminee: true, // Tache DEJA terminee
  ),
  Tache(
    id: '105',
    titre: 'Preparer le TP',
    description: 'Creer le repository Git',
    dateCreation: DateTime.now(),
  ),
];
```



```
        priorite: Priorite.haute,
    ),
];

// Boucle for-in pour ajouter chaque tache au gestionnaire
for (var tache in tachesDemo) {
    gestionnaire.ajouter(tache);
}

// -----
// BOUCLE DU MENU
// -----
bool continuer = true; // Variable de controle de la boucle

while (continuer) { // Tant que continuer == true, la boucle tourne

    // Afficher Le menu
    print('\n=====');
    print('== GESTIONNAIRE DE TACHES ==');
    print('=====');
    print('1. Ajouter une tache');
    print('2. Lister toutes les taches');
    print('3. Lister par priorite');
    print('4. Marquer comme terminee');
    print('5. Supprimer une tache');
    print('6. Rechercher');
    print('7. Statistiques');
    print('8. Quitter');
    print('=====');

    // Demander Le choix de l'utilisateur
    stdout.write('Votre choix : ');
    final choix = stdin.readLineSync(); // Lit l'entree clavier

    // Switch = branchement selon la valeur de choix
    switch (choix) {
        case '1':
            _ajouterTache(gestionnaire);
            break; // Sort du switch apres execution
        case '2':
            _listerToutes(gestionnaire);
            break;
```



```
case '3':
    _listerParPriorite(gestionnaire);
    break;
case '4':
    _marquerTerminee(gestionnaire);
    break;
case '5':
    _supprimerTache(gestionnaire);
    break;
case '6':
    _rechercher(gestionnaire);
    break;
case '7':
    _statistiques(gestionnaire);
    break;
case '8':
    continuer = false; // Met fin a la boucle while
    print('Au revoir !');
    break;
default:
    // Si le choix n'est pas entre 1 et 8
    print('Choix invalide. Veuillez entrer un nombre entre 1 et 8.');
```

```
}
}
}

// =====
// FONCTIONS AUXILIAIRES DU MENU
// =====

// -----
// Fonction pour ajouter une tache via le menu
// -----

void _ajouterTache(GestionnaireTaches gestionnaire) {
    stdout.write('ID : ');
    final id = stdin.readLineSync() ?? '';
    // ?? '' = si l'utilisateur appuie sur Entrée sans rien taper, id = ''

    stdout.write('Titre : ');
    final titre = stdin.readLineSync() ?? '';

    stdout.write('Description (optionnel) : ');
```



```
final description = stdin.readLineSync(); // Peut etre null

stdout.write('Priorite (basse/moyenne/haute) : ');
final prioriteStr = stdin.readLineSync()?.toLowerCase() ?? 'moyenne';
// ?.toLowerCase() = appel securise si non null
// ?? 'moyenne' = valeur par default si null

// Convertir la chaine en enum Priorite
Priorite priorite;
try {
    // firstWhere = trouve le premier element qui correspond
    // p.name == prioriteStr = compare "haute" avec "haute"
    priorite = Priorite.values.firstWhere((p) => p.name == prioriteStr);
} catch (e) {
    // Si la priorite est invalide, on utilise "moyenne"
    print('Priorite invalide, "moyenne" utilisee par default.');
```

```
    priorite = Priorite.moyenne;
}

// Creer la nouvelle tache
final tache = Tache(
    id: id,
    titre: titre,
    // Si la description est vide, on met null
    description: description?.isEmpty ?? true ? null : description,
    dateCreation: DateTime.now(), // Date de creation = maintenant
    priorite: priorite,
);

gestionnaire.ajouter(tache);
}

// -----
// Fonction pour lister toutes les taches
// -----
void _listerToutes(GestionnaireTaches gestionnaire) {
    final taches = gestionnaire.listerToutes();

    if (taches.isEmpty) { // isEmpty = verifie si la liste est vide
        print('Aucune tache enregistree.');
```

```
        return; // Sort de la fonction
    }
}
```



```
print('\n--- Toutes les taches (triees par date) ---');
// for-in = boucle sur chaque element de la liste
for (var tache in taches) {
    print(tache); // Appelle automatiquement toString()
}
}

// -----
// Fonction pour lister par priorite
// -----
void _listerParPriorite(GestionnaireTaches gestionnaire) {
    stdout.write('Priorite (basse/moyenne/haute) : ');
    final prioriteStr = stdin.readLineSync()?.toLowerCase() ?? 'moyenne';

    Priorite priorite;
    try {
        priorite = Priorite.values.firstWhere((p) => p.name == prioriteStr);
    } catch (e) {
        print('Priorite invalide.');
        return;
    }

    final taches = gestionnaire.listerParPriorite(priorite);

    if (taches.isEmpty) {
        print('Aucune tache avec la priorite "${priorite.name}");
        return;
    }

    print('\n--- Taches de priorite ${priorite.name.toUpperCase()} ---');
    for (var tache in taches) {
        print(tache);
    }
}

// -----
// Fonction pour marquer une tache comme terminee
// -----
void _marquerTerminee(GestionnaireTaches gestionnaire) {
    stdout.write('ID de la tache a marquer : ');
    final id = stdin.readLineSync() ?? '';
}
```



```
    gestionnaire.marquerCommeTerminee(id);
}

// -----
// Fonction pour supprimer une tache
// -----
void _supprimerTache(GestionnaireTaches gestionnaire) {
    stdout.write('ID de la tache a supprimer : ');
    final id = stdin.readLineSync() ?? '';
    gestionnaire.supprimer(id);
}

// -----
// Fonction pour rechercher
// -----
void _rechercher(GestionnaireTaches gestionnaire) {
    stdout.write('Terme a rechercher : ');
    final terme = stdin.readLineSync() ?? '';
    final resultats = gestionnaire.rechercher(terme);

    if (resultats.isEmpty) {
        print('Aucun resultat pour "$terme".');
        return;
    }

    print('\n--- Resultats pour "$terme" ---');
    for (var tache in resultats) {
        print(tache);
    }
}

// -----
// Fonction pour afficher les statistiques
// -----
void _statistiques(GestionnaireTaches gestionnaire) {
    final toutes = gestionnaire.listerToutes();
    final terminees = gestionnaire.listerTerminees();
    final enCours = gestionnaire.compterEnCours();

    print('\n--- STATISTIQUES ---');
    print('Total des taches : ${toutes.length}'); // ${} = interpolation avec expres
sion
```



```
print('Taches terminees : ${terminees.length}');  
print('Taches en cours : $enCours');  
print('Par priorite :');  
  
// Boucle sur chaque valeur de L'enum  
for (var p in Priorite.values) {  
    final count = gestionnaire.listerParPriorite(p).length;  
    print('  ${p.name.toUpperCase()} : $count');  
}  
}
```

QUESTIONS DE COMPREHENSION GLOBALES

Question	Reponse
Q1 : Quel est le role de la fonction <code>main()</code> ?	C'est le point d'entree du programme. C'est la premiere fonction executee.
Q2 : Que fait <code>stdin.readLineSync()</code> ?	Elle lit une ligne de texte saisie par l'utilisateur dans le terminal.
Q3 : Pourquoi utiliser <code>while (continuer)</code> ?	Pour repeter le menu jusqu'a ce que l'utilisateur choisisse de quitter.
Q4 : A quoi sert <code>switch (choix)</code> ?	A executer differentes actions selon la valeur de <code>choix</code> .
Q5 : Que fait <code>break</code> dans un switch ?	Il sort du switch pour eviter d'executer les autres cas.
Q6 : Pourquoi les fonctions auxiliaires commencent par <code>_</code> ?	Le <code>_</code> les rend privees (accessibles uniquement dans ce fichier).
Q7 : Que fait <code>try/catch</code> ?	<code>try</code> = essaie d'executer le code. <code>catch</code> = attrape une erreur si elle survient sans planter le programme.



COMMENT EXECUTER

powershell

Dans le dossier ou se trouve le fichier

dart run tp1_gestionnaire_taches.dart

RESUME DES NOTIONS UTILISEES

Notion	Syntaxe	Usage
Enum	<code>enum Priorite { basse, moyenne, haute }</code>	Definir des valeurs fixes
Classe	<code>class Tache { }</code>	Modeliser un objet
Constructeur	<code>Tache({required this.id})</code>	Creer une instance
Factory	<code>factory Tache.fromJson()</code>	Creer depuis JSON
Null safety	<code>String?, ??, ?.</code>	Gerer les valeurs null
Methodes fonctionnelles	<code>where(), sort(), add()</code>	Manipuler des listes
Boucle while	<code>while (continuer) { }</code>	Repetier tant que vrai
Boucle for-in	<code>for (var t in liste) { }</code>	Parcourir une liste
Switch	<code>switch (choix) { case '1': ... }</code>	Branchement multiple



Notion	Syntaxe	Usage	18
Try/catch	<code>try { } catch (e) { }</code>	Gérer les erreurs	